

# Report 10 – Confronto QALS - GUROBI

September 25, 2026

# 1 Limiti Gurobi

Per quanto riguarda i limiti dell'ottimizzatore Gurobi, nella documentazione ufficiale si può notare come, oltre alle limitazioni imposte dalle possibili licenze, l'unico reale limite è relativo alla potenza del dispositivo che si sta usando. Ne conviene che non risulta essere adeguato definire un confronto in relazione a ciò, ma piuttosto è necessario discutere in relazione al tempo impiegato. Sulla base dei seguenti risultati dati dalla media di 10 run:

| N. items | QALS (s) | Gurobi (s) |
|----------|----------|------------|
| 72       | 22.42    | 21.83      |
| 73       | 21.71    | 21.40      |
| 74       | 19.73    | 30.53      |
| 75       | 23.12    | 25.12      |
| 76       | 22.47    | 77.87      |
| 77       | 24.4     | 88.4       |

Table 1: Confronto dei tempi di esecuzione tra QALS e Gurobi al variare del numero di item.

si può concludere che non sempre vale la pena eseguire QALS: come visto nei risultati fino a 73 items (incluso) ci converrebbe utilizzare Gurobi, dal 74-esimo in poi QALS sembra impiegarci meno tempo.

## 2 Limiti Teorici Embedding

Si consideri ora un aspetto più quantitativo, ovvero cercare di capire quali siamo, in fisici, i limiti dell'annealer. Per fare ciò è necessario ricordare che fino ad ora si è considerata una mappatura 1:1 sulla topologia Pegasus con 5700 qubit teorici, utilizzabili 5436 ossia il 3% (ref: QALS\_implementations). Vien da se che nel momento in cui cercassi di eseguire QALS con un problema di 5437 items fallirebbe.

Se invece si utilizzasse `EmbeddingComposite` che, anziché fare un mapping 1:1, cerca di trovare un minor embedding, cioè un processo (euristico) di mappatura delle variabili di un problema di ottimizzazione (rappresentato come un grafo logico) sulla struttura e sulle connessioni fisiche (topologia) dei qubit di un computer quantistico, il problema principale che sarebbe che il minor embedding potrebbe fallire nel caso in cui il problema sia troppo denso.

Comunque utilizzare un modo e l'altro non cambia l'embedding massimo.

### 2.1 Ma allora quali sarebbero i vantaggi di usare EmbeddingComposite?

Utilizzando il costum il problema che si cerca di ottimizzare cambia ogni volta e quindi quello che si sta ottimizzando non è il vero problema ma una sua versione approssimata, con `EmbeddingComposite` invece no.

### 3 Limiti Reali Embedding

Nella pratica usando il simulated annealing:

```
1 def generate_pegasus(n):
2     G = dnx.pegasus_graph(16)
3
4     tmp = nx.to_numpy_array(G) # 5760 x 5760
5
6     rows = []
7     cols = []
8
9     for i in range(n):
10         rows.append(i)
11         cols.append(i)
12         for j in range(n):
13             if(tmp.item(i,j)):
14                 rows.append(i)
15                 cols.append(j)
16
17     return list(zip(rows, cols))
```

si definisce una matrice di 5760 qubits, tutti utilizzabili, il che crea una differenza di 324 qubits. Queste differenze portano grandi divergenze nel vero annealing perché di quei 324 qubits è importante anche la loro locazione nella topologia e non solo la quantità.